FREE STARTER KIT

Ship software without a dev team.

The exact files, rules and prompts I give Claude Code before every project — the setup behind a car-import CRM, a cross-posting tool and two AI assistants built in 60 days.

Sergey Lee-chan · Automotive Department Lead, ECOFACTOR
EV + AI · Ukraine → Poland

How to use this kit

Claude forgets everything between sessions. Everything below exists to fix that one problem — give it memory, rules and a clear goal, and it behaves like a senior teammate instead of a clever stranger.

1. The 5-minute setup

- Install Claude Code (desktop app or terminal).
- Create one folder per project.
- Put a **CLAUDE.md** file in it (template on page 3).
- Create a **docs/** folder for briefs and decisions (page 4).
- Start every session with the kickoff prompt (page 5).

2. What's inside

- p.3 — CLAUDE.md template: the project's "constitution"
- p.4 — Folder structure that gives AI long-term memory
- p.5 — 10 prompts I use on every project
- p.6 — Safety rules and the pre-launch checklist

One rule above all: you own the problem, AI owns the typing. Never ask it to "build an app". Ask it to solve one painful thing for one real person.

CLAUDE.md template

Claude reads this file automatically at the start of every session. Keep it under one page.

```
# Project: [name]

## Who it's for
[One person, one situation. E.g. "Sales manager who
handles 40 car-import leads a week in spreadsheets."]

## The problem
[What hurts, how often, what it costs in hours or money.]

## What "done" looks like
- [User can do X in under Y minutes]
- [No manual copy-paste between A and B]

## Rules
- Language of the UI: [English / Polish / Ukrainian]
- Keep it simple: no new libraries without asking me.
- Never store passwords or API keys in code — use .env.
- Before deleting or overwriting anything, show me first.
- After each change, tell me how to test it in one sentence.

## Decisions log
See docs/decisions.md — read it before proposing changes.

## Current status
[What works, what's next. Update at end of each session.]
```

Give AI a memory

This folder structure is how my projects survive between sessions — and how a new teammate (human or AI) gets up to speed in 30 seconds.

```
my-project/
├── CLAUDE.md          ← rules + status (auto-read)
├── docs/
│   ├── brief.md       ← who, what, why
│   ├── decisions.md   ← "we chose X because Y" (dated)
│   ├── questions.md   ← open questions for the owner
│   └── work-log.md    ← what was done each session
├── .env.example       ← names of secrets, never values
└── src/               ← the code (AI writes, you test)
```

decisions.md — example entry

```
## 2026-10-12 — Payments
Cash on delivery only for launch. Online payment
after 50 orders. Why: avoids 2 weeks of provider review.
```

Why it matters: without a decisions log, AI will happily "improve" things you already decided against. With it, every session starts where the last one ended.

10 prompts I use on every project

1. Kickoff
Read CLAUDE.md and docs/. Summarize the goal in 3 lines and list what you'd do first. Don't write code yet.

2. Ugly version first
Build the smallest version that solves [problem] for [person]. One screen. No design polish.

3. Explain like I'm the owner
Explain what you just built without technical words. What can break, and how would I notice?

4. Test plan
Give me a 5-step checklist to test this by hand.

5. Second opinion
Review your own change as a strict senior engineer. List bugs and security risks, most serious first.

6. Stuck
We've tried this twice. Stop. List 3 different approaches with pros and cons, then recommend one.

7. Decision record
Add today's decision to docs/decisions.md with the why.

8. Handover
Update CLAUDE.md status and work-log.md so a new session can continue tomorrow without me.

9. Cost check
What will this cost per month to run? Cheaper options?

10. Kill switch
What should NOT be automated here and why?

Safety rules & launch checklist

AI makes building cheap. It doesn't make mistakes cheap. These are the lines I don't cross alone.

Bring in a professional when you touch

- Payments and money flows
- Personal data of customers (GDPR)
- Logins, passwords, access to company accounts
- Anything that can delete data in bulk

Before you give it to real users

□ A colleague tested it without your help
□ Secrets live in .env, not in the code
□ There's a backup and you've restored it once
□ Prompt #5 (second opinion) found nothing serious
□ You know who fixes it if it breaks on a Sunday
□ You measured the "before": hours or money it costs today

Found this useful? I share what I build with AI — costs, failures and prompts — plus EV reality from Ukraine and Poland. Follow me on LinkedIn: **Sergey Lee-chan**. Reply to my DM and tell me what you're building — I read every one.